

DATA ANALYTICS

FASE 3 | AULA 02 -

MODELO ENTIDADE RELACIONAMENTO (MER)

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	31
O QUE VOCÊ VIU NESTA AULA?	32
REFERÊNCIAS	33

EMANIP

O QUE VEM POR AÍ?

Bem-vindo e bem-vinda à segunda aula do nosso curso, onde você explorará a poderosa ferramenta de modelagem de dados: o “Modelo Entidade-Relacionamento” (ou MER). Nesta aula, vamos além do básico, examinando como o MER pode ser aplicado em arquiteturas de dados modernas, como Data Warehouses, Data Lakes, Data Lakehouses, Data Mesh e Data Fabric. Nosso objetivo é que você compreenda não apenas o conceito fundamental do MER, mas também como ele pode ser adaptado para o contexto da big data, onde volumes gigantescos de dados e diferentes tipos de armazenamento exigem estratégias de modelagem inteligentes e escaláveis.

Prepare-se para uma aula que combina teoria, prática e exemplos reais de aplicação. Vamos discutir como cada uma dessas arquiteturas pode ser beneficiada pelo uso do MER e como aplicar esse modelo para resolver problemas específicos de estruturação de dados. Ao final, você terá uma visão completa de como o MER é adaptado e utilizado em ambientes de dados inovadores, ampliando suas possibilidades de carreira e entendimento do mercado atual de big data.

HANDS ON

Primeiro, vamos construir um MER básico em um banco de dados relacional, que servirá como ponto de partida para os outros exemplos. Nesse exemplo, criaremos um banco de dados relacional para gerenciar informações de clientes, produtos e vendas.



SAIBA MAIS

Serviço AWS: Amazon Relational Database Service (RDS)

A Amazon RDS permite criar bancos de dados relacionais gerenciados com suporte a PostgreSQL, MySQL, Oracle, SQL Server, MariaDB, entre outros. Usamos o MER para definir tabelas de dados e relacionamentos em uma estrutura relacional.

Passo a Passo

Definir as Entidades e Atributos:

- Entidades principais: cliente, produto, venda.
- Relacionamentos: um cliente pode fazer várias vendas; cada venda contém um ou mais produtos.

Criação das Tabelas no SQL:

- Crie uma instância RDS no console da AWS.
- Escolha o mecanismo de banco de dados desejado (por exemplo, MySQL ou PostgreSQL).
- Conecte-se ao banco de dados usando um cliente SQL e execute os comandos SQL para criar tabelas e definir chaves primárias e estrangeiras.

Este MER básico modela um banco de dados relacional que serve como estrutura fundamental. Agora, vamos adaptar esse modelo para cada uma das arquiteturas de armazenamento avançadas.

DIAGRAMA MER

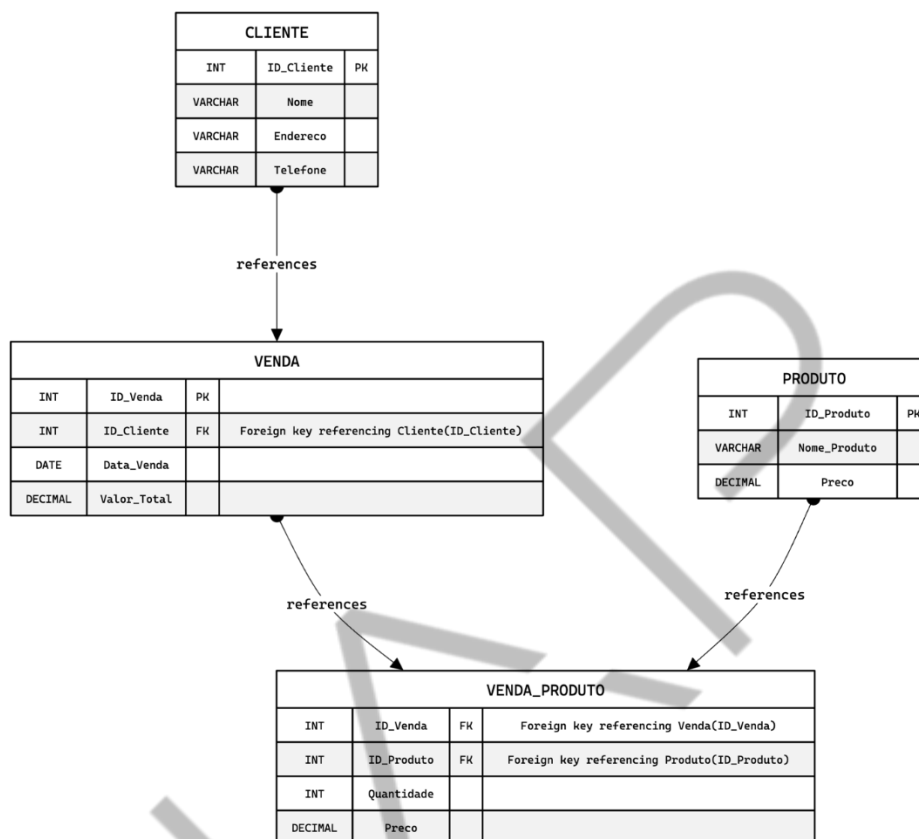


Figura 1 - Diagrama MER
 Fonte: elaborado pelo autor (2025)

Estrutura das Entidades e Relacionamentos

Entidade Cliente:

- **ID_Cliente**: chave Primária (PK) que identifica unicamente cada cliente.
- **Nome**: nome do cliente.
- **“Endereco”**: endereço do cliente.
- **Telefone**: telefone de contato do cliente.

Cada cliente pode ter várias vendas associadas, estabelecendo uma relação 1com a tabela Venda.

Entidade Produto:

- **ID_Produto**: chave Primária (PK) que identifica unicamente cada produto.

- **Nome_Produto:** nome do produto.
- **“Preço”:** preço do produto.

Um produto pode estar associado a várias vendas por meio da tabela **Venda_Produto**, formando uma relação N com a entidade **Venda**.

Entidade Venda

- **ID_Venda:** chave Primária (PK) que identifica unicamente cada venda.
- **ID_Cliente:** chave Estrangeira (FK) que referencia **ID_Cliente** na entidade **Cliente**, indicando o cliente que realizou a venda.
- **Data_Venda:** data em que a venda foi realizada.
- **Valor_Total:** valor total da venda.

Relaciona-se com a tabela **Cliente** por meio de uma chave estrangeira, formando uma relação 1 (um cliente pode ter várias vendas).

Entidade Venda_Produto

- **ID_Venda:** parte da Chave Primária composta (PK) e Chave Estrangeira (FK) que referencia **ID_Venda** na tabela **Venda**.
- **ID_Produto:** parte da Chave Primária composta (PK) e Chave Estrangeira (FK) que referencia **ID_Produto** na tabela **Produto**.
- **Quantidade:** quantidade de produtos vendidos nesta venda específica.
- **“Preço”:** preço unitário do produto no momento da venda.

Esta tabela serve como intermediária para implementar a relação **N** entre **Venda** e **Produto**, permitindo que cada venda contenha múltiplos produtos e que cada produto seja vendido em múltiplas vendas.

Resumo dos Relacionamentos

Cliente → **Venda**: relação 1.

Cada cliente pode ter várias vendas associadas.

Venda → **Venda_Produto**: relação 1

Cada venda pode incluir vários produtos, e a quantidade de cada produto é armazenada em **Venda_Produto**.

Produto → Venda_Produto: relação N

Por meio de Venda_Produto, um produto pode estar presente em várias vendas e cada venda pode conter vários produtos.

-- Tabela Cliente

```
CREATE TABLE Cliente (  
  ID_Cliente INT PRIMARY KEY,  
  Nome VARCHAR(100),  
  Endereco VARCHAR(255),  
  Telefone VARCHAR(15)  
);
```

-- Tabela Produto

```
CREATE TABLE Produto (  
  ID_Produto INT PRIMARY KEY,  
  Nome_Produto VARCHAR(100),  
  Preco DECIMAL(10, 2)  
);
```

-- Tabela de Relacionamento Venda

```
CREATE TABLE Venda (  
  ID_Venda INT PRIMARY KEY,  
  ID_Cliente INT,  
  Data_Venda DATE,  
  Valor_Total DECIMAL(10, 2),  
  FOREIGN KEY (ID_Cliente) REFERENCES Cliente(ID_Cliente)  
);
```


-- Tabela de Relacionamento Venda_Produto (associação entre Venda e Produto)

```
CREATE TABLE Venda_Produto (  
  ID_Venda INT,  
  ID_Produto INT,  
  Quantidade INT,  
  PrecoDECIMAL(10, 2),  
  PRIMARY KEY (ID_Venda, ID_Produto),  
  FOREIGN KEY (ID_Venda) REFERENCES Venda(ID_Venda),  
  FOREIGN KEY (ID_Produto) REFERENCES Produto(ID_Produto)  
);
```

Código-fonte 1 - Create Table:Tabela Relacional
Fonte: elaborado pelo autor (2025)

Integração com Data Warehouse (DW)

O Data Warehouse é uma evolução da base relacional, focando em dados históricos e análises. Aqui, criaremos tabelas de fatos e dimensões a partir do MER básico.

Passo a Passo

Definir Fatos e Dimensões:

- **Fato:** venda.
- **Dimensões:** cliente, Produto, Data.
- **Serviço AWS:** Amazon Redshift

Amazon Redshift é um serviço de Data Warehouse que permite armazenar e analisar grandes volumes de dados históricos. Ele suporta modelagem em estrela, usando tabelas de fatos e dimensões.

1. Crie um cluster Redshift no console da AWS.

- 2. Conecte-se ao cluster usando ferramentas como SQL Workbench ou o console de consultas da AWS.
- 3. Implemente o MER com tabelas de fatos e dimensões para uma estrutura de dados eficiente para consultas analíticas. Utilize o “Esquema em ESTRELA” (ou STAR Schema):

Esquema em ESTRELA (STAR Schema):

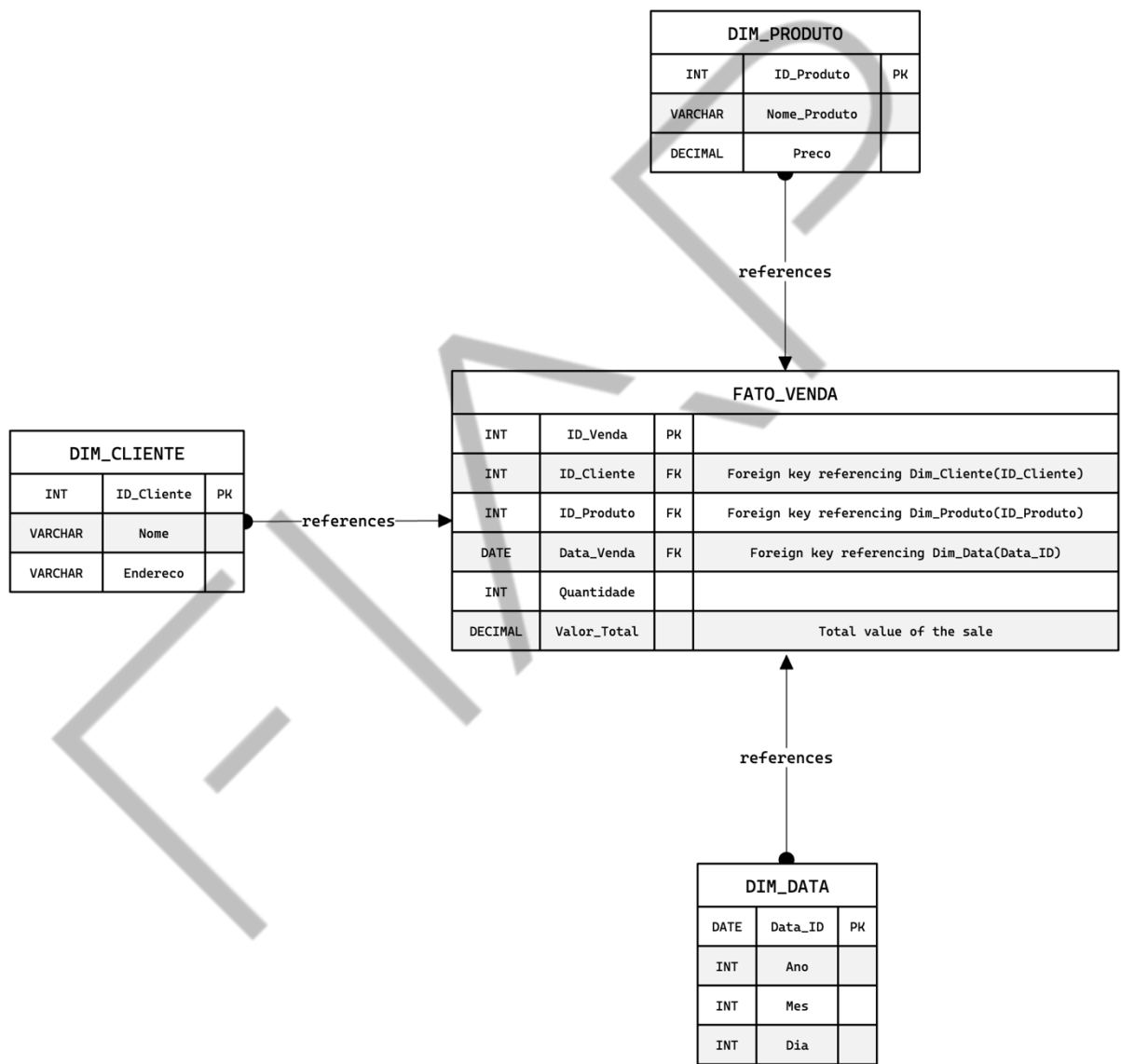


Figura 2 - Esquema em ESTRELA
Fonte: elaborado pelo autor (2025)

Esquema em Floco de Neve (Snowflake Schema):

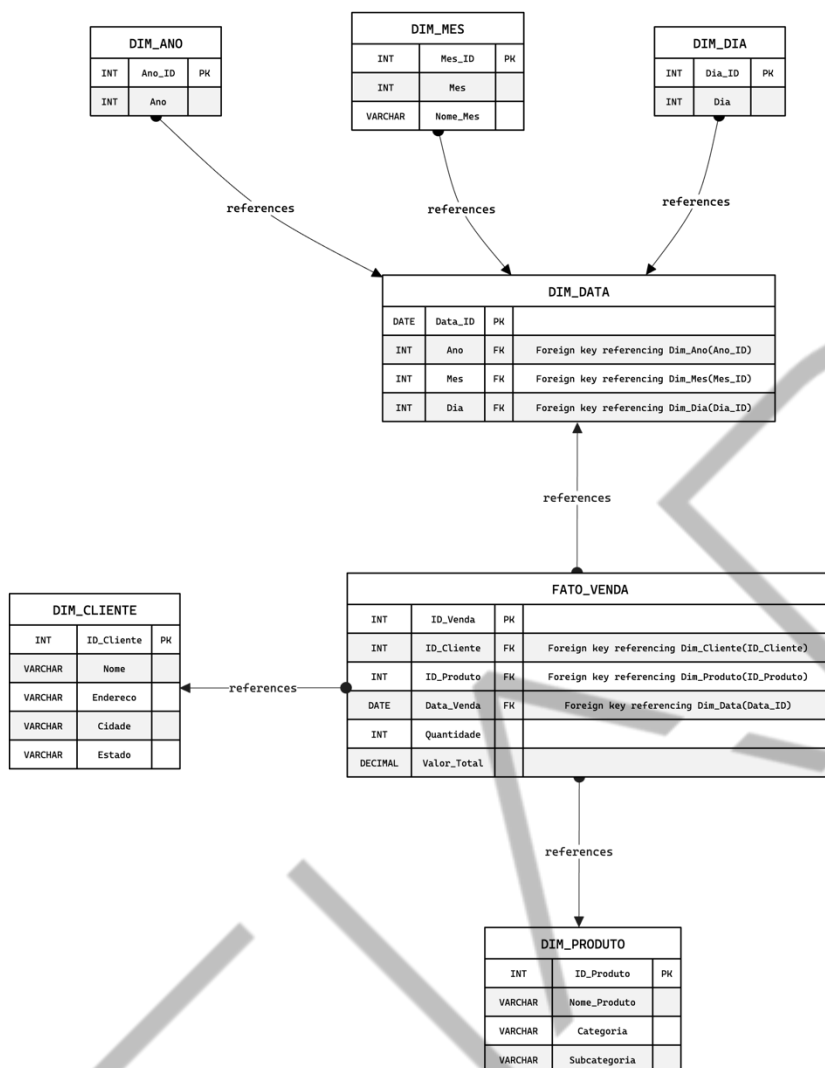


Figura 3 - Esquema em Floco de Neve
Fonte: elaborado pelo autor (2025)

Explicação do Diagrama

Este diagrama representa a estrutura do Data Warehouse com uma tabela de fato e tabelas de dimensão associadas.

Tabela de Fato - Fato_Venda

- **ID_Venda:** chave Primária (PK) que identifica unicamente cada registro de venda.
- **ID_Cliente:** chave Estrangeira (FK) que referencia a tabela Dim_Cliente.
- **ID_Produto:** chave Estrangeira (FK) que referencia a tabela Dim_Produto.

- **Data_Venda:** chave Estrangeira (FK) que referencia a tabela Dim_Data.
- **Quantidade:** número de unidades vendidas.
- **Valor_Total:** valor total da venda.

A tabela de fato armazena eventos de vendas e se conecta às tabelas de dimensão para capturar detalhes sobre clientes, produtos e datas.

Tabela de Dimensão - Dim_Cliente

- **ID_Cliente:** chave Primária (PK) que identifica unicamente cada cliente.
- **Nome:** nome do cliente.
- **Endereco:** endereço do cliente.

Representa os detalhes do cliente para análise de vendas.

Tabela de Dimensão - Dim_Produto

- **ID_Produto:** chave Primária (PK) que identifica unicamente cada produto.
- **Nome_Produto:** nome do produto.
- **Categoria:** categoria do produto.

Armazena detalhes dos produtos para análises de vendas e inventário.

Tabela de Dimensão - Dim_Data

- **Data_ID:** chave Primária (PK) que identifica unicamente cada data.
- **Ano:** ano da venda.
- **Mes:** mês da venda.
- **Dia:** dia da venda.

Permite análise temporal das vendas, segmentando por ano, mês e dia.

Resumo dos Relacionamentos

- **Dim_Cliente → Fato_Venda:** relacionamento 1, onde cada cliente pode estar associado a várias vendas.
- **Dim_Produto → Fato_Venda:** relacionamento 1, onde cada produto pode estar associado a várias vendas.

- **Dim_Data** → **Fato_Venda**: relacionamento 1, permitindo análise de vendas ao longo do tempo.

Tipos de Schemas

Esquema em ESTRELA (STAR Schema): no Esquema em ESTRELA, temos a tabela **Fato_Venda** no centro, diretamente conectada às **tabelas de dimensão Dim_Cliente, Dim_Produto e Dim_Data**.

Este formato é simples e eficaz para consultas rápidas, ideal para agregações e sumarizações. As tabelas de dimensão contêm detalhes diretos e completos, sem subdimensões, o que permite uma estrutura compacta e de fácil acesso.

Esquema em Floco de Neve (Snowflake Schema): no Esquema em Floco de Neve, cada tabela de dimensão pode ser normalizada em subdimensões, criando uma estrutura ramificada.

Dim_Cliente possui atributos adicionais como **Cidade** e **Estado** e **Dim_Produto** inclui subcategorias, enquanto **Dim_Data** tem **Nome_Mes**.

Esta estrutura melhora a normalização e economiza espaço de armazenamento, pois evita duplicação de dados. No entanto, o acesso aos dados é mais complexo e pode impactar a performance de consultas em comparação com o esquema em estrela.

Código SQL:

```
-- Tabela de Fato

CREATE TABLE Fato_Venda (

ID_Venda INT PRIMARY KEY,

ID_Cliente INT,

ID_Produto INT,

Data_Venda DATE,

Quantidade INT,

Valor_Total DECIMAL(10, 2),

FOREIGN KEY (ID_Cliente) REFERENCES Dim_Cliente(ID_Cliente),
```

```
FOREIGN KEY (ID_Produto) REFERENCES Dim_Produto(ID_Produto),
FOREIGN KEY (Data_Venda) REFERENCES Dim_Data(Data_ID)
);

-- Tabelas de Dimensão

CREATE TABLE Dim_Cliente (
ID_Cliente INT PRIMARY KEY,
Nome VARCHAR(100),
Endereco VARCHAR(255)
);

CREATE TABLE Dim_Produto (
ID_Produto INT PRIMARY KEY,
Nome_Produto VARCHAR(100),
Categoria VARCHAR(50)
);

CREATE TABLE Dim_Data (
Data_ID DATE PRIMARY KEY,
Ano INT,
Mes INT,
Dia INT
);
```

Código-fonte 2 - Create Table: Tabela Multidimensional
Fonte: elaborado pelo autor (2025)

Essa estrutura em estrela permite análises eficientes, facilitando consultas e relatórios sobre vendas por cliente, produto ou data.

Integração com Data Lake (DL)

No Data Lake, armazenamos dados brutos, mantendo a flexibilidade para análise de grandes volumes de dados. Usaremos o MER para definir um catálogo e metadados para facilitar a organização.

Passo a Passo

Serviço AWS Amazon Simple Storage Service (S3).

O Amazon S3 é ideal para armazenar dados brutos em um Data Lake, permitindo flexibilidade no armazenamento de dados estruturados e não estruturados.

1. Crie buckets S3 para armazenar dados brutos.
2. Organize os dados em pastas (prefixos de S3) para diferentes categorias (por exemplo, clientes, vendas...).
3. Use o **AWS Glue Data Catalog** para criar um catálogo de dados e definir metadados, facilitando a consulta.

DIAGRAMA DATA LAKE

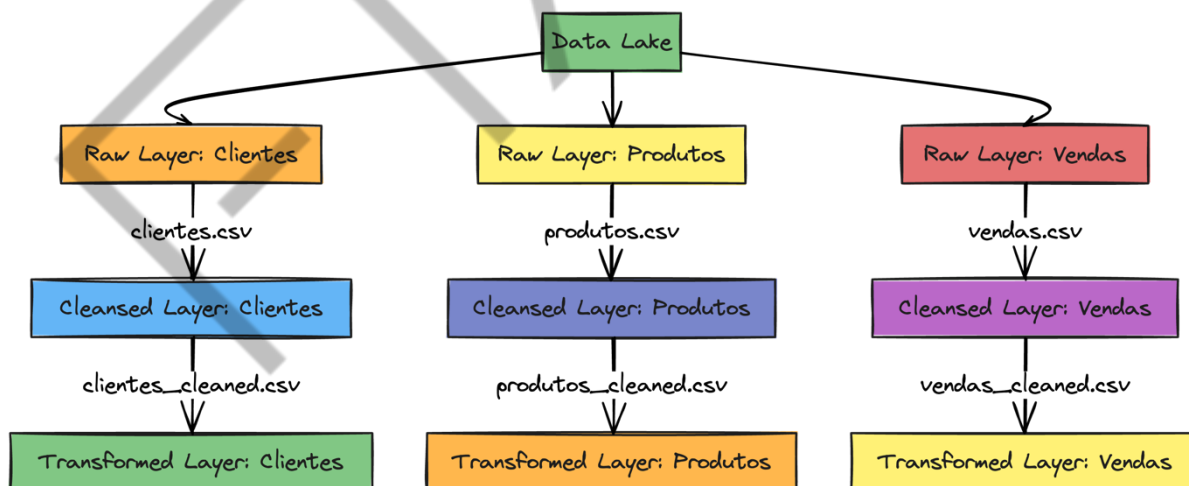


Figura 4 – Diagrama Data Lake
Fonte: elaborado pelo autor (2025)

Camadas de Dados

Um Data Lake é geralmente dividido em várias camadas de dados, cada uma representando um estágio do ciclo de vida dos dados:

- **Raw Layer (ou “Camada Bruta”)**: contém os dados não processados exatamente como são recebidos de fontes externas. As tabelas de **clientes, produtos e vendas** estão armazenadas em formato bruto.
- **Cleansed Layer (ou “Camada Limpa”)**: contém dados que foram processados para corrigir erros, remover duplicatas ou aplicar regras de qualidade.
- **Transformed Layer (ou “Camada Transformada”)**: contém dados que foram modelados para serem consumidos por processos analíticos ou relatórios, normalmente seguindo uma estrutura organizada e agregada.
- **Curated Layer (ou “Camada Curada”)**: inclui dados altamente organizados e prontos para consumo em dashboards, relatórios ou modelos de machine learning.

Modelo de Diretórios no Data Lake

Dentro de cada camada, os dados podem ser organizados por ano, mês, dia, tipo de dado ou qualquer outra estrutura hierárquica que facilite o acesso.

Exemplo de Estrutura de Diretórios no S3:

- `/raw/clientes/ano=2024/mes=11/dia=11/clientes.csv`.
- `/cleansed/produtos/ano=2024/mes=11/dia=11/produtos.csv`.
- `/transformed/vendas/ano=2024/mes=11/dia=11/vendas.csv`.

Catálogo de Dados

Para facilitar o uso e a descoberta dos dados, o Data Lake pode ser integrado com um Data Catalog (por exemplo, o AWS Glue Data Catalog), que organiza metadados e define esquemas para os arquivos armazenados, permitindo consultas SQL diretamente sobre os dados no S3.

Processos de ETL e ELT

Para transformar os dados entre camadas, é comum usar processos de **ETL (Extract, Transform, Load)** ou **ELT (Extract, Load, Transform)**:

- **ETL**: extrai dados da camada bruta, transforma-os e os carrega na camada curada.

- **ELT**: extrai e carrega diretamente na camada bruta ou limpa, e as transformações ocorrem à medida que os dados são consumidos.

Exemplo de Modelagem Expandida

Um diagrama para este modelo expandido incluiria:

- Uma estrutura de camadas (Raw, Cleansed, Transformed, Curated).
- Conexões representando o fluxo de dados entre as camadas.
- Diretórios ou buckets S3 organizados para cada camada.
- Um componente de Data Catalog para o mapeamento dos dados e metadados.

Exemplo de código bash para criar uma estrutura:

```
# 1. Criação de diretórios no S3 para armazenar dados nas diferentes camadas

aws s3api create-bucket --bucket meu-datalake-raw

aws s3api create-bucket --bucket meu-datalake-cleansed

aws s3api create-bucket --bucket meu-datalake-transformed


# 2. Organização dos arquivos CSV na camada Raw (dados brutos)

aws s3 cp clientes.csv s3://meu-datalake-raw/clientes/ano=2024/mes=11/dia=11/clientes.csv

aws s3 cp produtos.csv s3://meu-datalake-raw/produtos/ano=2024/mes=11/dia=11/produtos.csv

aws s3 cp vendas.csv s3://meu-datalake-raw/vendas/ano=2024/mes=11/dia=11/vendas.csv


# 3. Organização dos arquivos na camada Cleansed após limpeza e validação dos dados

aws s3 cp clientes_cleaned.csv s3://meu-datalake-
```

```
cleansed/clientes/ano=2024/mes=11/dia=11/clientes_cleaned.csv

aws      s3      cp      produtos_cleaned.csv      s3://meu-datalake-
cleansed/produtos/ano=2024/mes=11/dia=11/produtos_cleaned.csv

aws      s3      cp      vendas_cleaned.csv      s3://meu-datalake-
cleansed/vendas/ano=2024/mes=11/dia=11/vendas_cleaned.csv

# 4. Organização dos arquivos na camada Transformed para dados prontos
para análise

aws      s3      cp      clientes_transformed.csv      s3://meu-datalake-
transformed/clientes/ano=2024/mes=11/dia=11/clientes_transformed.csv

aws      s3      cp      produtos_transformed.csv      s3://meu-datalake-
transformed/produtos/ano=2024/mes=11/dia=11/produtos_transformed.csv

aws      s3      cp      vendas_transformed.csv      s3://meu-datalake-
transformed/vendas/ano=2024/mes=11/dia=11/vendas_transformed.csv
```

Código-fonte 3 - Código Criação de Diretórios
 Fonte: elaborado pelo autor (2025)

Integração com Data Lakehouse (DLH)

No Data Lakehouse, unimos o armazenamento de dados brutos com tabelas estruturadas. Usaremos processos ETL para transformar dados e carregá-los na estrutura final.

Usando Amazon S3 + Redshift Spectrum

Serviço AWS: Amazon S3 + Amazon Redshift Spectrum

O Data Lakehouse combina dados brutos e estruturados. No Amazon S3, armazenamos dados brutos, enquanto o Redshift Spectrum permite consultas SQL diretamente no S3, integrando esses dados ao Data Warehouse Redshift.

Diagrama

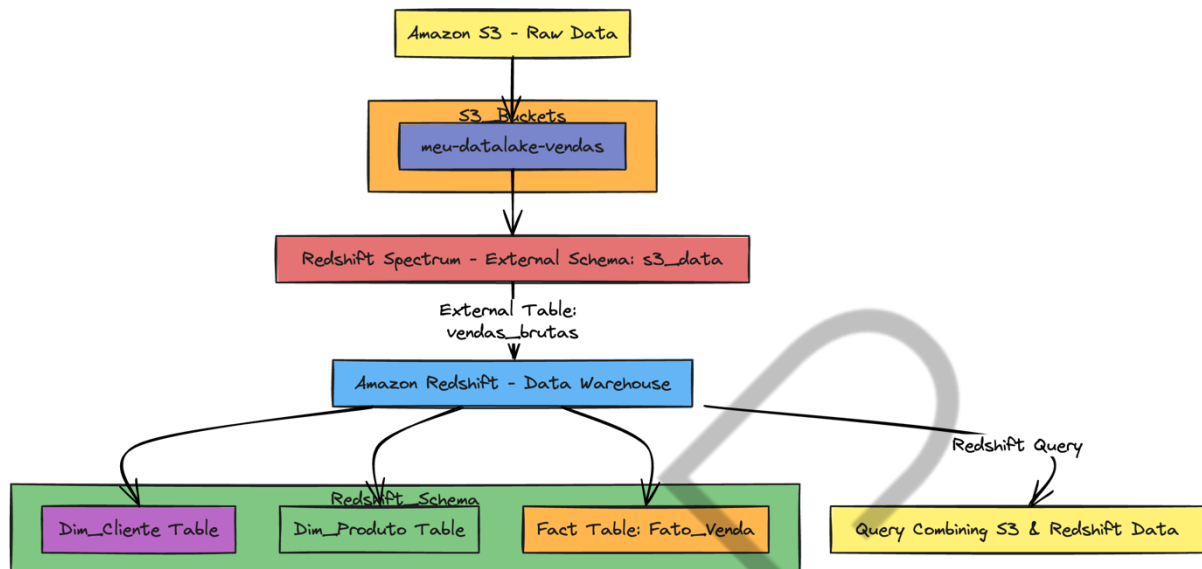


Figura 5 – Diagrama
Fonte: elaborado pelo autor (2025)

Estrutura do Diagrama

Amazon S3 - Raw Data: o bucket meu-datalake-vendas armazena dados brutos que podem ser consultados diretamente.

Redshift Spectrum – External Schema (s3_data): define o esquema externo para dados no S3, permitindo que o Amazon Redshift acesse a tabela vendas_brutas como uma tabela externa.

Amazon Redshift - Data Warehouse: inclui tabelas dimensionais Dim_Cliente e Dim_Produto, além de uma tabela Fato_Venda.

Consultas podem combinar dados da tabela externa em S3 com dados estruturados no Redshift.

Consulta de Dados:

A consulta final utiliza uma combinação de dados brutos e estruturados, unindo Redshift Spectrum e Amazon Redshift para obter insights.

Passo a Passo:

1. Armazene dados brutos no Amazon S3.
2. Use o AWS Glue para catalogar os dados brutos.

3. Conecte-se ao Redshift Spectrum e consulte os dados do S3, combinando com dados estruturados no Redshift.

```
-- Consulta dados brutos do S3 usando Redshift Spectrum

CREATE EXTERNAL SCHEMA s3_data
FROM DATA CATALOG
DATABASE 'meu_datalake'
IAM_ROLE 'arn:aws:iam::123456789012:role/MyRedshiftRole'
CREATE EXTERNAL DATABASE IF NOT EXISTS;

-- Tabela externa no S3 (dados brutos)
CREATE EXTERNAL TABLE s3_data.vendas_brutas (
  ID_Venda INT,
  ID_Cliente INT,
  Data DATE,
  Valor_Total DECIMAL(10,2)
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION 's3://meu-datalake-vendas/';

-- Consultar dados brutos e estruturados simultaneamente
SELECT v.ID_Venda, c.Nome, p.Nome_Produto, v.Valor_Total
FROM s3_data.vendas_brutas v
JOIN Dim_Cliente c ON v.ID_Cliente = c.ID_Cliente
JOIN Dim_Produto p ON v.ID_Produto = p.ID_Produto;
```

Código-fonte 4 – Consulta de dados

Fonte: elaborado pelo autor (2024)

Data Mesh com AWS (Divisão por domínios usando AWS Glue e Amazon Redshift)

No Data Mesh, os dados são organizados em domínios independentes. Na AWS, podemos utilizar AWS Glue e Amazon Redshift para catalogar e estruturar os dados de cada domínio, permitindo consultas distribuídas e descentralizadas.

Exemplo:

Crie domínios como "Vendas" e "Marketing", com tabelas independentes, que podem ser acessadas por meio de AWS Glue e Redshift.

Diagrama Data Mesh

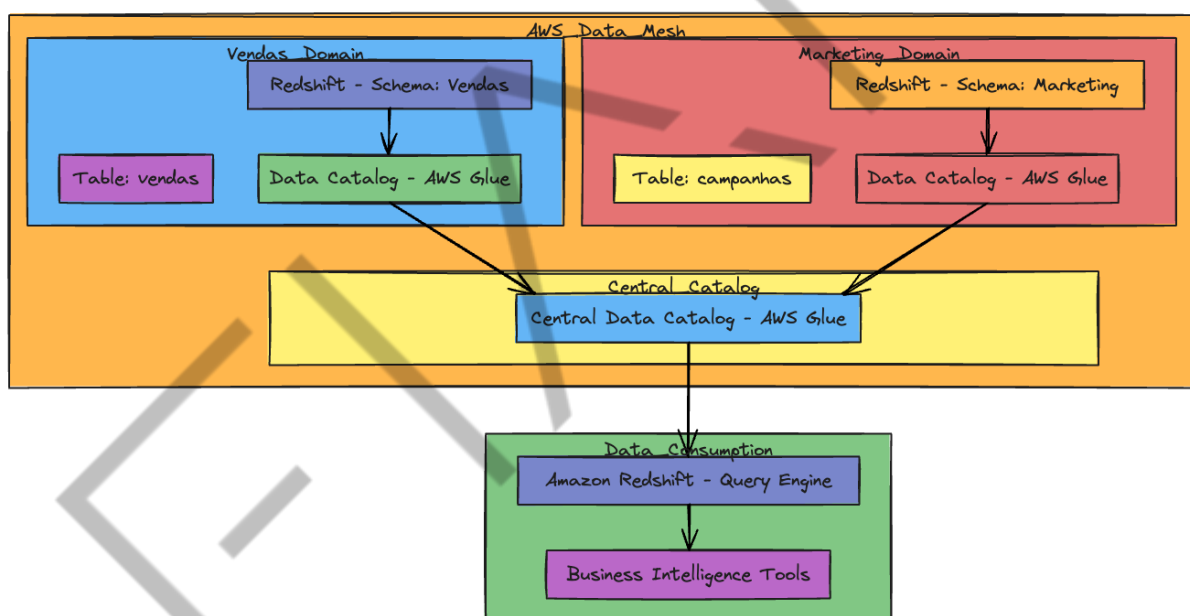


Figura 6 - Diagrama Data Mesh
Fonte: elaborado pelo autor (2025)

Estrutura do Diagrama

Domínios de Dados:

- **Vendas Domain:** contém o esquema **Vendas** no Redshift com a tabela vendas e o catálogo de dados do **AWS Glue** para o domínio de vendas.

- **Marketing Domain:** inclui o esquema **Marketing** no Redshift com a tabela campanhas e o catálogo de dados do **AWS Glue** para o domínio de marketing.

Central Catalog:

- Um **Catálogo de Dados Central** no AWS Glue agrega os catálogos de cada domínio, facilitando a descoberta e o acesso aos dados entre domínios.

Data Consumption:

- **Amazon Redshift Query Engine** permite a consulta dos dados integrados a partir do catálogo central.
- **Business Intelligence Tools** (ferramentas de BI) consomem os dados para análise e relatórios.

Passos no AWS Glue e Amazon Redshift:

1. Configure catálogos independentes no AWS Glue para cada domínio (Vendas, Marketing).
2. Carregue dados de cada domínio em clusters de Redshift diferentes ou em schemas separados dentro do mesmo cluster.

-- RedshiftSchema para Vendas

```
CREATE SCHEMA vendas;  
  
CREATE TABLE vendas.vendas (  
  ID_Venda INT PRIMARY KEY,  
  ID_Cliente INT,  
  Data DATE,  
  Valor_Total DECIMAL(10, 2)  
);
```

-- Redshift Schema para Marketing

```
CREATE SCHEMA marketing;  
  
CREATE TABLE marketing.campanhas (  
  
ID_Campanha INT PRIMARY KEY,  
  
Nome_Campanha VARCHAR(100),  
  
Tipo VARCHAR(50)  
  
);
```

Código-fonte 5 – Vendas
Fonte: elaborado pelo autor (2025)

Data Fabric com AWS (AWS Data Exchange, AWS Glue e Amazon QuickSight)

O Data Fabric fornece uma camada unificada de dados, integrando diferentes fontes de dados em tempo real. Na AWS, o AWS Data Exchange facilita o acesso a dados de terceiros, o AWS Glue integra metadados, e o Amazon Quick Sight permite a visualização unificada dos dados.

Diagrama

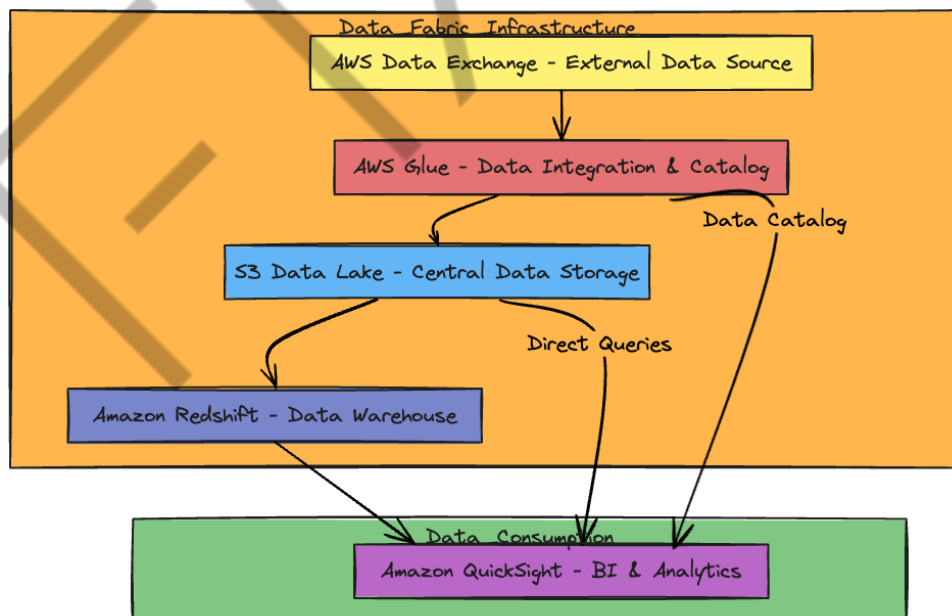


Figura 7 – Data Fabric
Fonte: elaborado pelo autor (2025)

Use o AWS Data Exchange para importar dados de terceiros e integre-o com dados internos para criar uma visão unificada no Amazon Quick Sight.

Vamos construir um Data Fabric na AWS, criando uma camada unificada de dados que integre informações de diversas fontes em tempo real, e que permita a visualização em Amazon QuickSight. Utilizaremos os seguintes serviços da AWS:

1. **AWS Data Exchange:** para acessar dados de terceiros e armazená-los em um bucket S3.
2. **AWS Glue:** para catalogar os dados e disponibilizá-los para consulta com Amazon Athena.
3. **Amazon Quicksight:** para visualizar os dados unificados.

Etapa 1: importar dados externos com AWS Data Exchange e Amazon S3.

Primeiro, vamos assinar um dataset do AWS Data Exchange e fazer o download dos dados para um bucket no Amazon S3.

Passo 1.1: inscrever-se em um Dataset no AWS Data Exchange:

1. No console do AWS Data Exchange, procure um dataset que você deseja utilizar. Exemplos incluem dados de mercado financeiro, demográficos ou meteorológicos.
2. Selecione o dataset desejado e clique em “**Subscribe**” (Inscrever-se).

Passo 1.2: exportando o Dataset para o S3:

Depois de assinado, configure a exportação dos dados para um bucket S3. Vamos utilizar o AWS CLI para esta tarefa.

```
# Exemplo de comando CLI para exportar o dataset
```

```
awsdataexchange start-job \  
  --type EXPORT_ASSETS_TO_S3 \  
  --details '{  
    "exportAssetsToS3": {  
      "dataSetId": "DATASET_ID",  
      "revisionId": "REVISION_ID",
```



```
"assetDestinations": [  
  {  
    "assetId": "ASSET_ID",  
    "bucket": "meu-bucket-datalake",  
    "key": "dados-externos/mercado-financeiro.csv"  
  }  
]  
}
```

Código-fonte 6 - Exemplo de comando CLI para exportar o dataset
Fonte: elaborado pelo autor (2025)

Substitua *"DATASET_ID"*, *"REVISION_ID"* e *"ASSET_ID"* com os valores específicos do dataset escolhido. Esse comando “salvará” o arquivo no bucket *meu-bucket-datalake*, na pasta *dados-externos/*.

Etapa 2: catalogar os dados com AWS Glue.

Agora que os dados estão armazenados no Amazon S3, vamos usar o AWS Glue para catalogá-los. Isso facilita a consulta dos dados com Amazon Athena.

Passo 2.1: criando um Crawler no AWS Glue:

1. No console do AWS Glue, vá em **“Crawlers”** e selecione **“Add Crawler”** (“Adicionar Crawler”).
2. Dê um nome ao seu crawler, como por exemplo *“crawler-dados-externos”*.
3. Configure o crawler para apontar para o bucket S3 onde os dados foram salvos (*s3://meu-bucket-datalake/dados-externos/*).

Passo 2.2: executar o Crawler:

Após configurar o Crawler, execute-o para que ele detecte o esquema dos dados e crie uma tabela no Glue Data Catalog.

Exemplo CLI para criar e executar o Crawler:

```
# Criando o crawler

aws glue create-crawler \

    --name "crawler-dados-externos" \

    --role "arn:aws:iam::123456789012:role/GlueCrawlerRole" \

    --database-name "meu-database" \

    --targets S3Targets=[{Path="s3://meu-bucket-datalake/dados-externos/"}]


# Executando o crawler

aws glue start-crawler --name "crawler-dados-externos"
```

Código-fonte 7 - Exemplo CLI para criar e executar o Crawler:
Fonte: elaborado pelo autor (2025)

Etapa 3: consultando os dados com Amazon Athena.

Com os dados catalogados, podemos usar o Amazon Athena para consultas SQL.

1. No console do Amazon Athena, selecione o banco de dados que foi criado pelo Crawler do Glue (*meu-database*).
2. Execute a consulta SQL abaixo para visualizar os dados:

```
SELECT * FROM meu_database.dados_externos LIMIT 10;
```

Essa consulta retornará os primeiros 10 registros do dataset externo que foi importado e catalogado.

Etapa 4: visualizando os dados com Amazon Quicksight.

Agora, vamos conectar o Amazon Quicksight ao Glue Data Catalog para criar visualizações e dashboards dos dados unificados.

Passo 4.1: conectando o Quicksight ao Glue Data Catalog:

1. No console do Amazon Quicksight, vá para Manage Data Sources e selecione Athena como a fonte de dados.
2. Escolha o banco de dados do Glue que contém a tabela catalogada (*meu-database*).
3. Selecione a tabela *dados_externos* e crie um dataset em Quicksight.

Passo 4.2: criando visualizações no Quicksight:

Após importar o dataset, você pode criar diferentes visualizações (como por exemplo, gráficos de linha, colunas ou tabelas dinâmicas) para visualizar os dados de forma unificada.

1. No Amazon Quicksight, vá em “New Analysis” (ou Nova Análise).
2. Selecione o dataset *dados_externos* que você criou.
3. Escolha o tipo de visualização desejado e configure as métricas e dimensões com base nos dados do dataset.

Exemplo de visualização em Quicksight:

Depois de configurar o dataset em Quicksight, você pode criar gráficos, tabelas e outros tipos de visualizações para representar insights do dataset. Você pode criar, por exemplo:

- Gráfico de Linha para observar tendências de mercado ao longo do tempo.
- Tabela Dinâmica para verificar um resumo dos valores financeiros por categoria.

Resumo dos Passos:

1. **AWS Data Exchange:** importe o dataset e salve-o no Amazon S3.
2. **AWS Glue:** configure e execute um crawler para catalogar os dados no Glue Data Catalog.
3. **Amazon Athena:** realize consultas SQL sobre os dados catalogados.
4. **Amazon Quicksight:** conecte-se ao Glue Catalog para criar visualizações e dashboards unificados.

Esses passos criam uma arquitetura Data Fabric na AWS, permitindo que você integre dados de várias fontes e visualize-os em tempo real em uma única interface.

Integrando o Modelo Entidade-Relacionamento em diferentes arquiteturas (base relacional, Data Warehouse, Data Lake, Data Lakehouse, Data Mesh e Data Fabric), vemos como adaptar e organizar os dados para atender diferentes necessidades de armazenamento e análise. Cada contexto requer uma abordagem específica do MER, e o código fornecido serve como um ponto de partida prático para aplicar esses conceitos em cada ambiente.

Conceitos Fundamentais do MER

O Modelo Entidade-Relacionamento (ou MER) é uma ferramenta gráfica que facilita a organização de dados em sistemas de informação. Ele é composto por três elementos principais:

Entidades: representam elementos ou objetos que têm relevância dentro de um sistema, como "Cliente", "Produto" e "Pedido".

Atributos: são as propriedades das entidades. Exemplos incluem "nome" e "idade" para um cliente ou "preço" para um produto.

Relacionamentos: associam as entidades entre si, como o relacionamento de "Cliente" e "Pedido", indicando que clientes fazem pedidos.

Estrutura de Data Warehouse com MER

No Data Warehouse (DW), o MER é ajustado para organizar dados históricos e estruturados. No DW, são criadas tabelas de fatos e dimensões para armazenar dados que precisam de estrutura organizada. Essa divisão permite que informações de diferentes áreas (como vendas e inventário) possam ser unidas, facilitando relatórios e análises complexas.

Exemplo prático: para um sistema de varejo, uma tabela de fatos pode ser "Venda", enquanto as dimensões incluem "Cliente", "Produto" e "Data".

Diagrama: crie um diagrama MER representando entidades como "Produto" e "Cliente" e seus relacionamentos com "Venda".

Estrutura de Data Lake com MER

O Data Lake (DL) difere do Data Warehouse por armazenar dados em seu formato bruto e não-estruturado. O MER, portanto, é menos aplicado diretamente, mas o uso de catálogos de dados e metadados torna a organização e recuperação mais fácil. O MER serve para organizar um catálogo de dados que facilita a recuperação e uso dos dados não estruturados.

Exemplo prático: em uma plataforma de streaming, um Data Lake poderia armazenar dados brutos de comportamento do usuário, como histórico de visualização e cliques. Esses dados são organizados posteriormente para gerar insights.

Diagrama: use o MER para estruturar um catálogo de dados, com entidades como “Usuário”, “Conteúdo” e “Ação”.

Estrutura de Data Lakehouse com MER

O Data Lakehouse é uma arquitetura híbrida que combina o Data Warehouse e o Data Lake. Ele oferece flexibilidade para armazenar dados brutos e organizados, permitindo análises complexas e estruturadas. No MER, o Data Lakehouse é representado tanto por tabelas estruturadas quanto por entidades de dados brutos, o que exige a criação de relacionamentos híbridos.

Exemplo prático: uma empresa de logística pode armazenar dados brutos de sensores de transporte, enquanto mantém dados organizados de clientes e entregas para relatórios gerenciais.

Diagrama: organize um MER para representar os dados brutos e estruturados, mostrando como eles se conectam no Lakehouse.

Estrutura de Data Mesh com MER

O Data Mesh distribui a responsabilidade pelos dados entre os domínios, criando uma estrutura descentralizada. Cada domínio gerencia seus próprios dados como “produtos”, o que requer sub-diagramas MER para cada domínio. O MER é aplicado para garantir consistência e integridade dos dados, enquanto mantém uma visão unificada entre os domínios.

Exemplo prático: em uma organização global, o domínio “Marketing” e o domínio “Vendas” têm seus próprios dados. O MER ajuda a garantir que as duas áreas possam se comunicar e cruzar informações.

Diagrama: crie sub-diagramas MER para cada domínio e mostre como eles se interconectam.

Estrutura de Data Fabric com MER

O Data Fabric unifica fontes de dados em uma infraestrutura conectada. Isso permite o acesso aos dados em tempo real, independentemente de onde estão armazenados. O MER, nesse contexto, organiza os dados em uma camada que oferece fácil acesso e integridade de dados. Aplicações de IA no Data Fabric permitem que o MER seja automatizado para otimizar o fluxo de dados.

Exemplo prático: em uma rede de hospitais, o Data Fabric permite acessar dados de pacientes de qualquer unidade hospitalar. O MER organiza os dados para que todos os sistemas tenham fácil acesso às informações dos pacientes.

Diagrama: desenvolva um MER representando a estrutura conectada do Data Fabric, com entidades como “Paciente”, “Consulta” e “Hospital”.

MERCADO, CASES E TENDÊNCIAS

Nos últimos anos, o mercado de big data tem se expandido rapidamente, e as arquiteturas de dados estão cada vez mais complexas para atender às demandas modernas.

Tendências em Data Warehouses e Data Lakes

Data Warehouses e Data Lakes continuam a ser amplamente usados, especialmente em grandes corporações que precisam de dados históricos e brutos para análise e Machine Learning. A aplicação de MER nesses contextos garante que os dados sejam organizados de forma eficiente, facilitando a análise e a acessibilidade.

Cases de Sucesso

Amazon: a Amazon utiliza uma estrutura de Data Lakehouse para gerenciar dados de clientes, produtos e logística. O MER ajuda a garantir a organização desses dados, permitindo uma integração contínua entre as áreas.

Netflix: a Netflix aplica o Data Lake para armazenar dados de comportamento de usuários. Utiliza o MER em seus catálogos de dados para organizar e recuperar informações rapidamente, garantindo uma experiência personalizada.

Uber: utilizando Data Mesh, a Uber organiza dados de transporte em diferentes domínios, como “Motorista” e “Passageiro”. O MER contribui para a integração e consistência dos dados entre os domínios, permitindo que cada área tenha acesso aos dados relevantes.

“Fique de olho”:

A arquitetura Data Lakehouse está se tornando popular pela flexibilidade e escalabilidade, oferecendo uma alternativa promissora aos modelos tradicionais.

Data Mesh e Data Fabric são inovações que atendem à necessidade crescente de descentralização e unificação, promovendo acessibilidade em tempo real e organização por domínio.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, revisamos a estrutura e o funcionamento do Modelo Entidade-Relacionamento (MER) e exploramos como ele se aplica a diferentes arquiteturas de dados: Data Warehouse, Data Lake, Data Lakehouse, Data Mesh e Data Fabric. Você aprendeu sobre os conceitos fundamentais do MER e como adaptar essa estrutura para atender as demandas de cada arquitetura. Também conheceu cases de sucesso que ilustram o impacto dessas tecnologias no mercado e as tendências para o futuro do big data.

Próximos passos: relembre os conceitos abordados revisitando as videoaulas sempre que necessário e aplique o conhecimento adquirido nos exercícios práticos!

REFERÊNCIAS

ARTEMIS. **Data Lakes: Purposes, Practices, Patterns, and Platforms**. 2017. Disponível em: <https://artemis-research.com>. Acesso em: 26 mar. 2025.

DEHGHANI, Z. **Data Mesh: Delivering Data-Driven Value at Scale**. New York: O'Reilly Media, 2022.

GARTNER. **The Rise of the Data Fabric as the Foundation for Data Architecture**. 2021. Disponível em: <https://gartner.com>. Acesso em: 26 mar. 2025.

IBM Research. **Data Fabric Architecture and Best Practices**. IBM Journal of Research and Development, v. 64, 2022, p. 102-116. Disponível em: <https://ibm.com/research>. Acesso em: 26 mar. 2025.

INMON, W. **Data Lake Architecture: Designing the Data Lake and Avoiding the Garbage Dump**. New York: Technics Publications, 2016.

INMON, W. H. **Building the Data Warehouse**. New York: Wiley, 2005. TDWI. Data Warehousing and Business Intelligence. Business Intelligence Journal, v. 22, 2019. Disponível em: <https://tdwi.org>. Acesso em: 26 mar. 2025.

JOHN, T.; GOEL, S. **Data Lake for Enterprises**. New York: Packt Publishing, 2018.

KIMBALL, R.; ROSS, M. **The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling**. New York: Wiley, 2013.

VAN DER LANS, R. **Data Fabric: The Next Step in Data Integration**. New York: Technics Publications, 2021.

VINCENT, T.; SMITH, J. **Data Management for the Hybrid Cloud: Building a Unified Data Fabric**. San Francisco: McGraw-Hill, 2021.

ZBURIVSKY, D.; PARTNER, L. **Designing Cloud Data Platforms**. New York: O'Reilly Media, 2021.

PALAVRAS-CHAVE

Bancos de Dados. SQL. Modelagem de Dados

EXEMPLO

The background is a dark blue field filled with numerous small, light blue dots. Overlaid on this are several large, wavy, translucent lines in shades of blue, yellow, and red. These lines flow from the left side towards the right, creating a sense of motion. Scattered throughout the composition are various geometric shapes: a circle containing the number '7' in the upper center, a small circle on the left, a cross-like shape in the lower left, a small circle in the lower left, and a hexagon in the lower right.

POSTECH